```
$stmt->execute();
var_dump($stmt->fetchAll(PDO::FETCH_ASSOC));
```

```
Output:
array(1) {
  [0]=>
  array(1) {
    ["message"]=>
    string(14) "even more text"
  }
}
```

The key to this is `$pdo->prepare()` which returns a `PDOStatement`, and `$stmt->bindParam()`. The initial query contains a token `:id`, and `->bindParam(':id', $id)` substitutes that token for the value of `$id` in a safe way: any malicious code would be wrapped in special quotes by the time it reaches the DB, rendering it non-functional (you'll likely get errors or just empty results back). `->execute()` does what it says and executes the query on the DB server, and `->fetchAll()` returns the entire set of results in one go (as an associative array thanks to the `PDO::FETCH_ASSOC` argument). We've jumped ahead of ourselves a little here in that there are concepts of object access and variable use that we've yet to cover; you can always revisit this section once you've read the relevant chapters.

These are the bare essentials for database interaction but should be enough to get you started at a basic level of safety and security. `PDO` is pretty much the standard interface that most PHP code uses for DB interaction; you can find the documentation for it at https://www.php.net/manual/en/book.pdo.php

Summary

Hopefully after reading this chapter and following the examples, you now have a fully working PHP environment! Whether you are using Linux, Mac, or Windows, you should be able to at least run PHP interactively with `php -a` and follow along with the examples. You also have the option of accessing this book's code examples at https://github.com/paultregoing/ExploringPHP8, and this repository also contains Dockerfiles, which you can use to build images based on the public `php-cli` images. These docker images are what I have used to write this book and should contain all the necessary modules and configuration for things to work seamlessly.